Integrated Boards Tech Spec

Background

This document contains a description and inner workings of the Integrated Boards feature and its architecture. With this project, we're planning to address the improvements and shortcomings of the Focalboard project, at the same time that we develop a feature fully integrated with Mattermost, leveraging existing mechanisms like the information structure that the platform has, threads for conversations within the cards and the Property System Architecture to empower users to tailor the Boards workflows to their needs.

Vocabulary

- **Mattermost Information Structure:** this refers to the `system > team > channel` hierarchy that Mattermost as a platform uses to structure its data. In the context of permissions or data modeling, this references that the entities within Integrated Boards will fall under that structure, and will take advantage of it to reuse parts of the platform's domain logic.
- **Views:** a new entity, attached to a channel, that describes a way to represent that channel's data. Views are stored in the `views` table and have a `Type` field that determines their behavior and structure.
- **Board:** a view with `type='board'` that contains and organizes cards (posts with `type='card'`). A board uses a configurable property filter to define which cards from the channel are displayed within it, allowing multiple boards in the same channel to show different subsets of cards. Each board contains a collection of `subviews` that can be applied to present its filtered card data in different ways.
- **Subview:** a specific presentation configuration within a board view, stored in the view's table as a `JSONB` column. Each subview defines a presentation type (kanban, list, calendar, etc.), filters applied on top of the view's base card selection, which card properties to display, how cards are grouped and sorted, and other type-specific configuration details. Multiple subviews allow users to view the same set of cards through different lenses and organizational schemes.

Design Goals

1. Provide an overview of the architecture of the feature.
2. Detail the proposed data model and information flow from the store to the API.
3. Describe the required changes to the Property System Architecture for the implementation, and explain how the feature leverages the system to meet its requirements.

Scope

1. Enabling users to create, edit, and delete boards within a channel, where each board follows the channel's permission structure and membership, ensuring that only channel members can access the boards.
2. Enabling users to create custom property fields per board to adapt the workflow to their needs, following the Property System Architecture approach.
3. Extending the Property System Architecture to provide generic REST API endpoints and websocket events that enable features to interact with properties through a unified interface, shifting from group based organization to object type based organization and supporting property hierarchy across system, team, and channel levels.
4. Representing board data through multiple subview types (starting with kanban for MVP), where each subview can be configured with custom filters, sorting rules, and display preferences.
5. Allowing users to start thread conversations on each card that function identically to standard Mattermost message threads, with replies accessible from both the board view and the channel's RHS, following standard notification behavior.
6. Displaying card creation events in the channel message feed as rich formatted messages that link to the card in the parent board, providing visibility to all channel members and enabling quick navigation to the newly created card.
7. Enabling plugins to interact with board properties through an extended Plugin API that adapts to the Property System Architecture changes.

8. Enabling mobile users to access and interact with boards through dedicated mobile screens that support viewing boards, navigating cards, editing property values, and participating in card thread conversations.

Out of scope

- Team level properties
- Telemetry
- Shared Channels
- Elasticsearch indexing and search integration
- Import / export
- Data retention
- Board templates
- Card history, undo/redo
- Bulk operations in properties or cards
- CLI commands

Background Reading

- [Property System Architecture Technical Spec]
- [Integrated Boards proposal]
- [UX Decisions on the Property System]
- [Integrated Boards MVP (Figma)]

Design Summary

Integrated Boards is a feature that allows Mattermost users to create Boards to structure and present information as part of a channel. A board is always tied to a channel and follows the same permission and membership structure of it, leveraging the full potential of the current organizational structure that the platform already has.

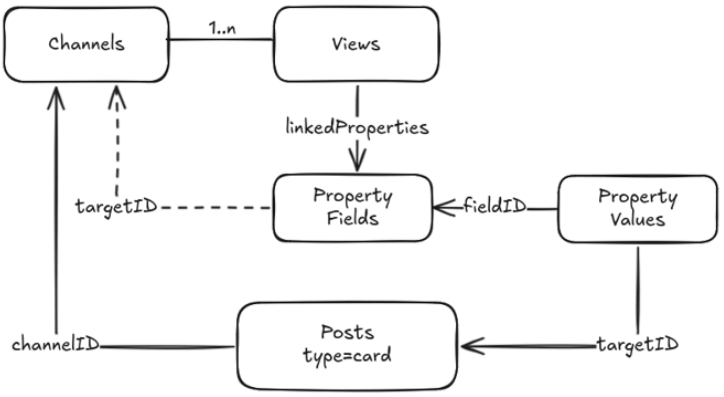
The cards within the boards are stored as `posts` in the database, and they have some particular behaviors and pieces of data connected to them:

- Cards are `posts` with the `type` set to `card`.
- When a new card is created, it is shown as part of the channel messages with its details.
- Every board has a property filter defined that determines which cards are part of it. There is a special system property `board` that is configured as the filter by default when a new board is created, but can be later removed from it by the user.

Besides the built-in `board` property, cards can have properties that users or admins create when using the feature. For the Integrated Boards MVP, those properties can be created at two levels:

- At the system level: so all the boards will be able to use them.
- At the channel level: boards themselves don't have properties, its channel has them, and users decide which boards within the channel use them. When a user creates a property on a board, they are really creating it on the channel and attaching it to the board. If another board in the same channel wants to create a property with the same name, the interface will offer the one that currently exists in the channel for the board to reuse it.

Board views will have different subviews configured to present the cards in different ways. For the MVP we're only going to cover the kanban subview, but the model has been designed taking the necessary flexibility into account, as every view will have specific configuration needs and fields.



The new `views` entity will come with its own database table, new REST API endpoints to access and manage it and websocket events for the API client to get updates when necessary, all of them described in detail further in this document. `subviews` will be part of the `views` table, stored as a `JSONB` field for flexibility, taking into account that currently the only type of view we will have is a board, but we'll expand the `views` usage in the future.

The Property System Architecture's new vision

The Integrated Boards feature is going to be implemented following the new Property System Architecture vision, which shifts from having the Property System being a flexible framework for other properties to mold and configure to being a feature on its own, with clearly defined limits on its behaviors and common entrypoints. Instead of having the feature implement endpoints, websocket events, permissions, etc. to use properties, the Property System will define mechanisms for this feature (and others) to use in a generic way.

As the system will require of some modifications to get to a point where Integrated Boards can fully use it in this new capacity, this document will cover as well the requirements, rationale and details on those changes, and at times will reference how the system used to work in an attempt to explain the reasoning behind a specific change.

Other Proposals

There are two main alternatives that were evaluated for this project, each related with a different area of the final data model. The first proposal was to avoid using the `posts` table to contain the cards, and create an independent `cards` table. Reusing the `posts` table for `cards` requires the table to be optimized for both use cases at the same time, can cause some performance issues and may require changes in the `posts` family of endpoints and application layer methods, to allow for callers to distinguish between `messages` and `cards`, that will be stored there. It means as well that API consumers will need to use the `posts` family of endpoints to manipulate cards, making the API for Integrated Boards a bit more difficult to understand, which was one of the complains we already faced with Focalboard.

On the other hand, creating the `cards` table would have required of a lot of implementation effort to make it work with key subsystems that we already have working with `posts`, like the possibility of creating threads of comments within the cards, the websocket subsystem for client updates and a completely new API, websockets and redux store sections in the codebase. It would have gone as well against the idea of using root posts as the "atom" of information in Mattermost, that is the vision of posts being a key piece in multiple features that, enriched with the Property System Architecture, allow users to adapt the platform to their needs, start threads everywhere and even mix information from different features at some point, thanks to the fact that all of them are modeled using the same base entities.

The other proposal that was taken into account was the idea of separating the model from the Mattermost information structure of system, team and channel. Implementations like Focalboard and Playbooks have gone back and forth on their entities (boards and playbooks) being tied to a channel, so a natural course of action when designing something from scratch would have been to try and provide with that flexibility of being able to have independent entities for the features to manage as they prefer.

The motivation to tie new features to this fixed structure is that we can build on top of the platform's model and, like in the case of using `posts` for cards, reuse plenty of systems that are already in place. The main two that quickly come to mind are the permissions and the websocket broadcast mechanisms, which we can directly reuse if all our entities and events belong to the whole system, a team or a channel. If we would have chosen to have independent boards, we would need independent permissions to those boards, membership structures and broadcast mechanisms to address board members as opposed of reusing everything that has already been built for channels.

Drawbacks / Tradeoffs

Overuse of the posts table

The `posts` table is one of the most heavily utilized tables (if not the most) in the whole platform, and has been extensively optimized for the channel messaging use case. Any modifications to its schema, indexes or query patterns require careful consideration due to the table's central role in platform performance.

Integrated Boards introduces a new use case for the `posts` table by storing cards as posts with `type = 'card'`. While this approach requires no immediate schema changes as the `type` field already exists in the `posts` table, it increases the volume and diversity of data in the table, potentially introducing different query patterns for card retrieval compared to message retrieval in the long run. As card usage scales, particularly in channels with many boards and cards, this could impact query performance for both card operations and traditional message operations.

Post type differences in the codebase

Introducing cards as a distinct post type may require updates throughout the codebase to differentiate between messages and cards where their handling diverges. Many existing methods, API endpoints and system processes currently assume all posts are messages, and while that's an advantage in terms of expanding what can be done with cards specifically and Integrated Boards as a whole, it may require modifications to explicitly filter by post type in some cases.

From extending API endpoints to return only elements of a given type, to allowing compliance exports to deal with posts and cards independently on its reports, each area that interacts with posts will need to be evaluated to determine whether it should handle all post types uniformly or apply type specific logic. While this is a significant effort, it can be treated as incremental work and addressed as specific use cases appear during development.

Property hierarchy constraint

One of the Property System Architecture's assumptions is that every property is going to be set at either a team, a channel or the system level. The consequence of this constraint is that features or plugins with entities outside this hierarchy will not be able to use the property system unless they align their models with it. While this constraint enables infrastructure reuse and simplifies implementation for channel based features, it reduces architectural flexibility for features with data models outside of it.

The new Property System Architecture vision requires all properties to be defined at the system, team, or channel level. This design constraint prevents features and plugins with entities outside the Mattermost information structure from using the generic property system unless they restructure their data models to align with the hierarchy. While this limitation enables infrastructure reuse and simplifies implementation for features that fit the channel-based model, it restricts architectural flexibility for features with alternative organizational structures.

Property System Architecture changes

Group Usage

For the initial vision of the Property System Architecture, property groups were both a way for a feature to specify that properties belonged to it, and the entrypoint for most if not all the services and queries, as it was intended that each feature would, in most cases, manage their own properties only.

The new Property System Architecture's vision uses the `ObjectType` as that logical entrypoint (instead of fetching say, Data Spillage properties, the client will fetch post properties, that include the Data Spillage ones and others), so the way we use groups is changing.

It will remain both for backwards compatibility reasons and because it will be useful as a filter when dealing with properties, but the current indexes and method signatures both internally and in the APIs will change to accommodate this new way of structuring the information.

A further step that we can take in this direction is to directly remove the `propertygroups` table in the database, and change the `GroupID` field by a `Group` field that contains directly the group name. The goal of the property groups table was to extend groups and make them configurable, to allow features to define a set of behaviors for all properties in a group. As the group is changing its meaning, we are free to remove that level of indirection and use group names directly, removing the need for clients to know specific group IDs and making APIs easier to use.

Permissions

The current approach for the Property System Architecture has every feature defining its own endpoints to modify its properties, so each feature can decide what permissions to check and how. With the new vision, we need a generic way to decide who is authorized to perform operations on both fields and values, that is independent by field.

To achieve this, each field will store (either as an attribute or a `permissions` column in the database) granular information about what users can do both on the field itself and on its values. Currently we have the following use cases to cover:

- Users can edit both the field and its values freely.
- Users can only add options to the field (for `select` and `multiselect` types), and then set its values freely. Only admins can edit the rest of the field.
- Only admins can edit the field, and users can set its values freely.
- Only admins can operate on the field, both editing and setting its values.

For audit purposes, we will need to track who has performed each one of these actions, either through our current auditing system, or by adding the `CreatedBy` and `UpdatedBy` columns to the `PropertyFields` and `PropertyValues` tables, so we can easily track these changes but use them as well as part of the information presented in the frontend.

Protected property fields

To be able to rely on the Property System Architecture, several features need to ensure that their properties are not going to be updated or deleted by users or an admin, so we need to account for this as a special case besides the ones listed above. These "protected properties" will not be able to be changed through the API, nor even by an admin, besides allowing admins or users to update the options for the `select` and `multiselect` types.

Examples of these properties would be the `status` or the `assignee` properties planned for the Integrated Boards feature, or several that the Data Spillage feature is using to flag and track post status. The feature should be the only one capable of modifying those properties through its codebase, knowing that the API will never allow users to tamper with them outside of its control.

This feature could be extended to be dynamic at some point if necessary, enabling features like ABAC for example to mark a property as protected when making it part of a policy rule, so no user or admin can update it as long as it's needed by the policy, and remove its protected status once it's not.

Introduction of the object type column

After the initial phase of properties usage, the philosophy behind them has changed, as well as the way we will use them. After the redesign, properties are now always attached to a specific object type (`post` properties, `channel` properties, etc) and that is the main grouping factor for them.

Following this new way of doing things, we need to introduce the `ObjectType` column to the property field table, which will indicate which object the property will apply to. This way, a property field for cards in a board will have this shape:

```
PropertyField{
    GroupID:    "IntegratedBoardsGroupID",
    Name:       "Rank",
    ObjectType: "post", // this is a post property, values will point to a post
    Type:       "select",
    TargetType: "channel", // property is set at a channel level
    TargetID:   "ChannelID"
}
```

Uniqueness

With this change of semantics comes as well a change on the way we define property uniqueness. Before, uniqueness was enforced by a database index that ensures that there were not two properties with the same name pointing to the same target in the same group.

```
CREATE UNIQUE INDEX IF NOT EXISTS idx_propertyfields_unique
  ON PropertyFields (GroupID, TargetID, Name) WHERE DeleteAt = 0;
```

After the change, properties can be created with a given name for a given `ObjectType` as long as there are no already existing properties with that name in their hierarchy, following the Mattermost information structure. This means that two different channels can have properties with the same name defined on them as that causes no collisions, but that a team and one of its channels cannot have two properties with the same name, and that a system level property forbids any team or channel creating properties with the same name.

Generic Property endpoints and websocket events

The feature will be built on top of the new Property System Architecture structure, that changes the way that features interact with properties. In the initial version of the Property System Architecture, each feature created their own application entrypoints to use properties, so they could control permissions and domain logic for every action performed over their properties.

With the new approach, properties work as an independent feature that other features can integrate with, but rather than having full control over how their properties work, features make use of the different options that the Property System provides.

As part of the Integrated Boards MVP, we will add a set of generic API endpoints and websocket events to the Property System Architecture. These changes will be described later in the REST API section of the document.

Existing features

Features like Data Spillage, User Attributes and Playbook Properties are already using the Property System Architecture with their own endpoints and domain logic. The new generic endpoints will use different entrypoints that bypass that logic, so they will check a blacklist and will not be able to reference or query any property field or value that belongs to the property groups of the protected features.

Data Model

The Integrated Boards feature builds on top of the Mattermost information structure (team, channel and post), and requires as well of an extra entity for its hierarchy, specifically Views. This feature will use views with `type='board'`, that we will refer to directly as “boards” through the document.

A channel can contain multiple boards, where each board defines a property filter with a set of conditions that determine which cards to display. The board will show only those cards that exist in the channel as posts with `type='card'` and satisfy its filter conditions. This way, we can have a team channel that contains a board to organize the team's workload, another one that tracks the candidates being interviewed for an open position, and a third one for the team's meeting agendas. All cards exist as posts within the channel, but each card will only appear on the boards whose filters it matches.

Each board can have multiple subviews, that describe the type of presentation that the information will have (a kanban, a list, a calendar...) and the specific configuration of the subview. Things like what filters are applied in the subview or the specific arrangement and sorting of the cards would be part of this configuration.

Database Schema

Posts

There are no changes specific to the post table. Card posts will have the following specifics:

- The `type` column will be equal to `card`, which is the main indicator of the being part of Integrated Boards.
- The post's `message` will contain the card's description.
- The post's `props` will contain information that is common to all cards, like the `title` or the `icon`.

Views

```sql
CREATE TABLE IF NOT EXISTS views (
  ID TEXT PRIMARY KEY,
  ChannelID VARCHAR(26) NOT NULL,
  Type TEXT,
  CreatorID TEXT NOT NULL,
  Title TEXT NOT NULL,
  Description TEXT,
  Icon VARCHAR(256),
  SortOrder INTEGER DEFAULT 0,
  Props JSONB,
  CreateAt BIGINT NOT NULL,
  UpadteAt BIGINT NOT NULL,
  DeleteAt BIGINT NOT NULL,
);
```

The views table will contain information about the view itself, its type (for this project, all views will have `type='board'`) details like the description and the icon and timestamps for the edits. It will have as well a `Props` `JSONB` column that will store parts of the view's configuration that are specific to each view type.

In the case of boards, as the properties for their cards are created at the channel level, we need a way to specify which properties are in use for each board, and that's done through `Props.LinkedProperties`, that contains a list of the property field IDs that are in use for the board.

Subviews

As part of the `Props` column, the views of type `board` will have a key called `subviews` that will contain a set of objects describing those subviews. Depending on the subview, the data that they will require to be configured is different, but we can expect common things like a `title`, its `type` (kanban, list, calendar, etc.), a `filter` to apply to the cards on top of the main view filter, a list of properties to show or hide, and then things that are specific to each view type, like the `groupBy` property for the kanban view, sorting information or operations to apply to list columns, for example.

Although the plan is to take advantage of the flexibility of the `Props` `JSONB` column for this initial implementation, there is potential for this part of the model to be normalized to a column of its own, or even further if we see that is beneficial as this feature or others that use `views` grow.

Properties Usage

For the dynamic pieces of information that users will be able to attach to the cards, the Integrated Boards feature uses properties to represent the card fields and its values. We distinguish two different types of properties: the feature properties

and the user properties.

Feature properties

The main feature property that we have is the `Boards` property. It is a platform-wide property that connects each card with its board, and it's later used as the starting point of multiple database queries that fetch boards contents.

Common details of the cards, like the `Title` or the `Icon` have been modeled as part of the `Post.Props` column of the database, but we could prefer to move them to be system properties of the feature as well.

User properties

Most of the properties that we will have are user properties, that users can create freely from the cards of a board. The properties are created at the channel level, and added to the list of properties that the board uses. Properties are unique, so a different board on the same channel cannot create another property with the same name, but the system will show the existing properties in the channel when the property is being created in case the user wants to reuse them.

Once the property hierarchy is implemented, these properties can be overridden by higher properties in the Mattermost information structure, so a team property with the same name of a channel property will take precedence over it, and a system property over team and channel properties.

Database queries

These are some of the most common operations for Integrated Boards and the queries they'd trigger:

- Get boards for a channel

```
SELECT * FROM views
        WHERE type = 'board'
          AND channelID = '{channel-id}';
```

- Get properties for a channel

```
SELECT * FROM propertyfields
        WHERE targetType = 'channel'
          AND targetID = '{channel-id}';
```

- Get posts for a board based on the `board` property value:

```
SELECT p.* FROM posts p
           JOIN propertyvalues pv_board
                ON pv_board.TargetID = p.ID
               AND pv_board.GroupID = '{boards-group-id}'
               AND pv_board.FieldID = '{board-field-id}'
               AND pv_board.Value = '{sample-board-id}'
          WHERE type = 'card'
            AND deleteAt = 0;
```

- Get posts for a board with a `severity=sev1` and `assignee=johndoe`:

```
SELECT p.* FROM posts p
           JOIN propertyvalues pv_severity
                ON pv_severity.TargetID = p.ID
               AND pv_severity.GroupID = '{boards-group-id}'
               AND pv_severity.FieldID = '{severity-field-id}'
               AND pv_severity.Value = 'sev1'
           JOIN propertyvalues pv_assignee
                ON pv_assignee.TargetID = p.ID
               AND pv_assignee.GroupID = '{boards-group-id}'
               AND pv_assignee.FieldID = '{assignee-field-id}'
               AND pv_assignee.Value = '{johndoe-user-id}'
          WHERE type = 'card'
            AND deleteAt = 0;
```

- Get posts for a board with a `severity=sev1`, `assignee=johndoe OR janedoe` and `status!=todo` filter:

```
SELECT DISTINCT p.* FROM posts p
                    -- severity = sev1
                    JOIN propertyvalues pv_severity
                         ON pv_severity.TargetID = p.ID
                        AND pv_severity.GroupID = '{boards-group-id}'
```

```
                        AND pv_severity.FieldID = '{severity-field-id}'
                        AND pv_severity.Value = 'sev1'
                    -- assignee = johndoe OR janedoe
                    JOIN propertyvalues pv_assignee
                         ON pv_assignee.TargetID = p.ID
                        AND pv_assignee.GroupID = '{boards-group-id}'
                        AND pv_assignee.FieldID = '{assignee-field-id}'
                        AND pv_assignee.Value IN ('{johndoe-user-id}', '{janedoe-user-id}')
                -- status is not todo. We left join with all status = todo
                -- and exclude them later with pv_status.TargetID IS NULL
                LEFT JOIN propertyvalues pv_status
                         ON pv_status.TargetID = p.ID
                        AND pv_status.GroupID = '{boards-group-id}'
                        AND pv_status.FieldID = '{status-field-id}'
                        AND pv_status.Value = '{todo-status-option-id}'
                    WHERE type = 'card'
                      AND deleteAt = 0
                      AND pv_status.TargetID IS NULL;
```

- Get property values from boards for posts with IDs:

```
SELECT * FROM propertyvalues
        WHERE groupID = '{boards-group-id}'
          AND targetID IN ({post-ids});
```

Permissions model

Integrated Boards leverages the Mattermost information structure to implement its permissions model, reusing existing channel permissions rather than introducing feature specific ones. Since boards are always tied to a channel, all board and card operations are gated by the channel's existing permission structure and membership rules. This means that users who have access to a channel automatically have access to its boards, and users without channel access cannot view or interact with those boards.

Card operations build upon existing post permissions but require additional flexibility beyond standard post behavior. Users with `create_post` permission in a channel can create cards in that channel's boards. However, unlike regular posts where users typically can only edit their own content, cards follow a more collaborative model where users with `create_post` permission can edit both their own cards and cards created by others within the same board. This divergence from standard post permissions necessitates special handling in the post edit and delete endpoints, where the `type='card'` check will bypass the usual ownership restrictions that apply to regular messages. Thread creation on cards follows standard Mattermost thread behavior, allowing any user who can reply to posts in a channel to create threads on cards in that channel's boards.

Property hierarchy

The Property System Architecture implements a hierarchical property model that aligns with the Mattermost information structure, allowing properties to be defined at the system, team, or channel level. This hierarchy is determined by a property field's `targetType` and `targetID` attributes, where system level properties have an empty `targetID`, team level properties reference a specific team, and channel level properties reference a specific channel. For the Integrated Boards MVP, properties will be created at either the system level (making them available to all boards across the platform) or at the channel level (making them available only to boards within that specific channel). Team level properties are explicitly out of scope for the initial implementation but the architecture supports their future addition.

Property name uniqueness is enforced within each branch of the hierarchy to prevent conflicts and ambiguity. The uniqueness constraint ensures that within a given object type, no two properties with the same name can exist at different levels of the same hierarchical path. For example, if a system level property named "Priority" exists for posts, no channel can create another post property named "Priority" because all channels inherit from the system level. However, two different channels that are not in the same hierarchical path can each have a channel level property named "Department" because they do not create a collision within their respective branches.

When users attempt to create a property that would violate the uniqueness constraint, the system will detect the collision and prevent the creation. For instance, if an admin has defined a system level property called "Rank" for posts, and a user

attempts to create a channel level property called "Rank" for their board's cards (which are posts), the system will reject the creation and suggest reusing the existing system level property instead. Similarly, if a channel already has a user defined "Rank" property and an admin later attempts to create a system level property with the same name, the system will prevent the creation to avoid breaking existing channel level properties. The user interface will handle these scenarios by showing existing properties with matching names during the creation flow, encouraging reuse over duplication and maintaining consistency across the property system.

REST API

There are two main blocks of new API endpoints, those specific to Integrated Boards and its entities, and the ones that are part of the Property System Architecture, which are generic but will be used by the Integrated Boards feature.

Integrated Boards endpoints

These are the new endpoints that the Integrated Boards feature will use for its own new entities. The payloads and responses correspond with the view with `type='board'` JSON representation:

- Create a new view for a channel

```
POST /api/v4/channels/{channel-id}/views
```

- List views for a channel

```
GET /api/v4/channels/{channel-id}/views
```

- Show a view

```
GET /api/v4/channels/{channel-id}/views/{view-id}
```

- Update a view

```
PATCH /api/v4/channels/{channel-id}/views/{view-id}
```

- Delete a view

```
DELETE /api/v4/channels/{channel-id}/views/{view-id}
```

Property System Architecture endpoints

These are the endpoints that will be added as part of the work on the Integrated Boards feature, but will be used for the Property System Architecture as generic entrypoints for all properties. The URL of these endpoints will have the `ObjectType` as one of its components, embracing the new Property System Architecture vision of organizing the property data around the different Mattermost objects.

- Create a new property field for a post

```
POST /api/v4/properties/posts/fields
```

- List post property fields

```
GET /api/v4/properties/posts/fields
```

- Update a post property field

```
PATCH /api/v4/properties/posts/fields/{field-id}
```

- Delete a post property field

```
DELETE /api/v4/properties/posts/fields/{field-id}
```

- List property values for a post

```
GET /api/v4/properties/posts/values/{post-id}
```

- Update property values for a post

```
PATCH /api/v4/properties/posts/values/{post-id}
```

The domain logic will make sure that the endpoint operations only affect to the properties of the type specified in the URL. Besides these endpoints, we will need to extend the search endpoints for the objects we add to the property system (only posts as part of this proposal) to enable filtering by property values.

Frontend

The frontend implementation for Integrated Boards follows a generic, reusable architecture that supports properties across multiple features while introducing specific structures for board and view management. The Redux store will be organized to reflect the Property System Architecture's structure, with dedicated sections for property fields and property values. These sections are designed to be feature agnostic, capable of storing properties from Integrated Boards or any other feature that uses the Property System Architecture. To facilitate efficient data retrieval, the implementation will include specialized selectors that filter properties by object type, group, etc., enabling components to access only the relevant subset of properties they need without processing the entire property collection.

Boards and views will have their own dedicated sections in the Redux store, following the already existing patterns for entity management. The board section will maintain the list of boards per channel, while the view section will store view configurations organized by board, allowing for efficient lookups and updates when users switch between views or modify view settings.

Websocket events

Views

One of the advantages of adhering to the Mattermost information structure is that everything under the Integrated Boards data model can reuse the mechanisms that we have to broadcast websocket events, so in case that (for example) a board is updated, we will broadcast that update event to all the members of the channel that the board is linked to.

Property System Architecture

Websocket events will have a generic format for properties, following the object type based organization. These events will be broadcast to the appropriate scope based on the property's target (system-wide, team specific, or channel specific), ensuring that only users with access to the relevant context receive property update notifications.

Mobile

The goal of Integrated Boards is to maintain feature parity between the web/desktop and the mobile implementations, from the MVP and through future iterations, so all the development phases will follow a mobile first approach.

Configuration

The feature will be implemented initially behind a Feature Flag, and the changes to the Property System Architecture will be fully compatible with the existing features, so they won't need to be protected in any way.

Plugins

Plugins will be able to interact with the Integrated Boards feature through multiple integration points. For property management, plugins can use the generic REST API endpoints and websocket events provided by the Property System Architecture following its new vision. Additionally, the Plugin API will be updated and extended to reflect the architectural shift from group based to object type based organization with enhanced filtering capabilities based on property attributes rather than solely on group membership.

For board and view management, the Plugin API will include new methods that allow plugins to manage boards and views within channels, enabling them to build integrations that extend or automate board workflows.

However, plugins face the same architectural constraint as other features regarding which entities a property can be attached to. Plugins cannot attach properties to their own custom entities unless those entities exist within the Mattermost

information structure (system, team, or channel hierarchy).

CLI

The CLI for Integrated Boards is out of the scope for this document, but in future efforts we could tackle the addition of new families of commands like `mmctl channel board` or `mmctl channel card` to manage the whole feature from the command line.

Migration

The migration from Focalboard to Integrated Boards is out of the scope of this document, but it will most probably be implemented through the Mattermost import system. It would need of adding the properties to the JSONL format that the platform currently uses for importing information, and transforming the Focalboard export into that format before ingesting it through the currently available entrypoints.

We should take into account when tackling this effort that Integrated Boards will not achieve feature parity with Focalboard after this first iteration, with things like card history or board templates not being part of the Integrated Boards feature set, so they should be either discarded or implemented as part of this effort.

Credits

Give credit here to anyone who helped you write the spec or provided feedback to improve it.

History

- 2nd October 2025: First draft of the document published.
- 9th October 2025: First version of the document published for review.
- 15th October 2025: Major update to restructure the data model around views.

Wording

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in RFC 2119